INTRODUCTION TO PROBLEM SOLVING
CSC 102

Makinde A .B.
Department of computer Science
Adeleke University Ede, Osun State.

Strategies

Ask questions!

- *What do I know about the problem?*
- *What is the information that I have to process in order the find the solution?*
- *What does the solution look like?*
- *What sort of special cases exist?*
- *How will I recognize that I have found the solution?*

Introduction to Problem Solving

Today, computers are all around us. We use them for doing various tasks in a faster and more accurate manner. For example, using a computer or smartphone, we can book train tickets online.It is only due to the use of computers that today, the booking of the train tickets has become easy. Online booking of train ticketshas added to our comfort by enabling us to book tickets from anywhere, anytime

We usually use the term computerisation to indicate the use of computer to develop software in order to automate any routine human task efficiently. Computers are used for solving various day-to-day problems and thus problem solving is an essential skill that a computer science student should know. It is pertinent to mention that computers themselves cannot solve a problem. Precise step-by-step instructions should be given by us to solve the problem. Thus, the success of a computer in solving a problem depends on how correctly and precisely we define the problem, design a solution (algorithm) and implement the solution (program) using a programming language

Computer science

Computer science is the study of the theoretical foundations of information and computation, and of practical techniques for their implementation and application in computer systems. So, computer science is all about taking in information and then performing some computations & analysis to solve a particular problem or produce a desired result, which depends on the application at hand.

Computer science is similar to mathematics in that both are used as a means of defining and solving some problem. In fact, computer-based applications often use mathematical models as a basis for the manner in which they solve the problem at hand.

A program,A programming language

A program is a sequence of instructions that can be executed by a computer to solve some problem or perform a specified task.

A programming language is an artificial language designed to automate the task of organizing and manipulating information, and to express problem solutions precisely

A programming language “boils down to” a set of words, rules and tools that are used to explain (or define) what you are trying to accomplish. There are many different programming languages just as there are many different "spoken" languages.

Traditional programming languages were known as structural programming languages (e.g., C, Fortran, Pascal, Cobol, Basic). Since the late 80's however, object-oriented programming languages have become more popular (e.g., JAVA, C++, C#)

Problem Solving

Regardless of the area of study, computer science is all about solving problems with computers. The problems that we want to solve can come from any real-world problem or perhaps even from the abstract world. We need to have a standard systematic approach to solving problems.

Since we will be using computers to solve problems, it is important to first understand the computer's information processing model. The model shown in the diagram below assumes a single CPU (Central Processing Unit). Many computers today have multiple CPUs, so you can imagine the above model duplicated multiple times within the computer.

Definition of Problem Solving

Problem solving is the process of identifying a problem, developing an algorithm for the identified problem and finally implementing the algorithm to develop a computer program.

Problem Solving is the sequential process of analyzing information related to a given situation and generating appropriate response options.

Programmers write instructions in various programming languages to perform their computation

tasks such as:

(i) Machine level Language

(ii) Assembly level Language

(iii) High level Language

Machine level Language

Machine code or machine language is a set of instructions executed directly by a computer's central processing unit (CPU). Each instruction performs a very specific task, such as a load, a jump, or an ALU operation on a unit of data in a CPU register or memory. Every program directly executed by aCPU is made up of a series of such instructions.

Assembly level Language

An assembly language (or assembler language) is a low-level programming language for a computer, or other programmable device, in which there is a very strong (generally one-to-one) correspondence between the language and the architecture's machine code instructions. Assembly language is converted into executable machine code by a utility program referred to as an assembler; the conversion process is referred to as assembly, or assembling the code.

High level Language

High-level language is any programming language that enables development of a program in much simpler programming context and is generally independent of the computer's hardware architecture.

High-level language has a higher level of abstraction from the computer, and focuses more on the programming logic rather than the underlying hardware components such as memory addressing and register utilization.

The first high-level programming languages were designed in the 1950s. Now there are dozens of different languages, including Ada , Algol, BASIC, COBOL, C, C++, JAVA, FORTRAN, LISP, Pascal, and Prolog. Such languages are considered high-level because they are closer to human languages and farther from machine languages. In contrast, assembly languages are considered low_x0002_level because they are very close to machine languages.

Procedure Oriented Programming Language

In the procedure oriented approach, the problem is viewed as sequence of things to be done such as reading , calculation and printing.

Procedure oriented programming basically consist of writing a list of instruction or actions for the computer to follow and organizing these instruction into groups known as functions.

Object Oriented Programing

“Object oriented programming as an approach that provides a way of modularizing programs by creating partitioned memory area for both data and functions that can be used as templates for creating copies of such modules on demand”.

Steps for problem Solvlng

Suppose while driving, a vehicle starts making a strange noise. We might not know how to solve the problem right away. First, we need to identify from where the noise is coming? In case the problem cannot be solved by us, then we need to take the vehicle to a mechanic. The mechanic will analyse the problem to identify the source of the noise, make a plan about the work to be done and finally repair the vehicle in order to remove the noise. From the above example, it is explicit that, finding the solution to a problem might consist of multiple steps.

When problems are straightforward and easy, we can easily find the solution. But a complex problem requires a methodical approach to find the right solution. In other words, we have to apply problem solving techniques. Problem solving begins with the precise identification of the problem and ends with a complete working solution in terms of a program or software. Key steps required for solving a problem using a computer are shown in Figure 4.1 and are discussed in following subsections.

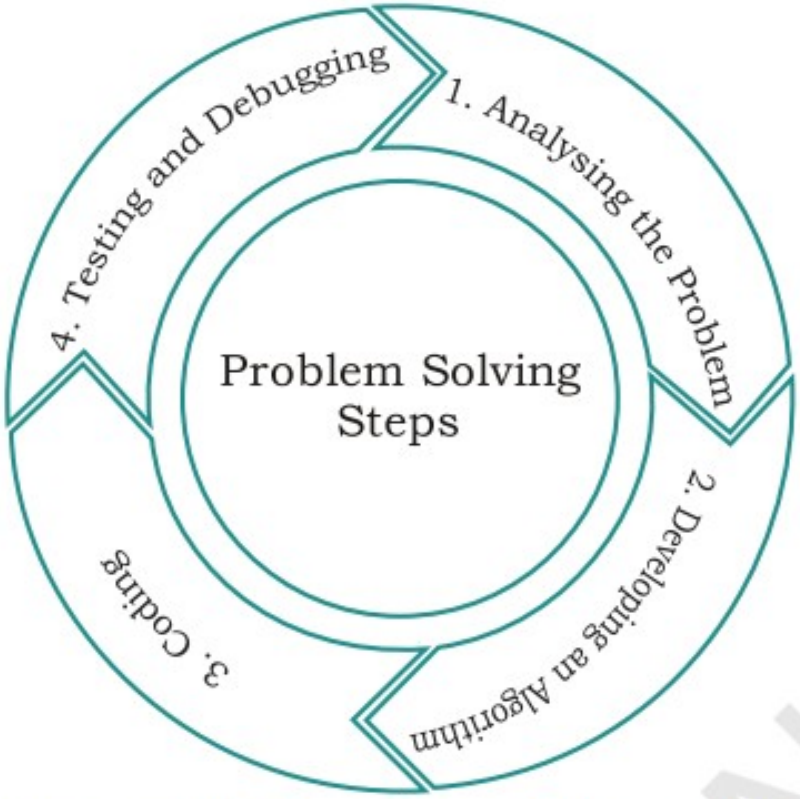


Figure 4.1: Steps for problem solving

Analysing the problem

It is important to clearly understand a problem before we begin to find the solution for it. If we are not clear as to what is to be solved, we may end up developing a program which may not solve our purpose. Thus, we need to read and analyse the problem statement carefully in order to list the principal components of the problem and decide the core functionalities that our solution should have. By analysing a problem, we would be able to figure out what are the inputs that our program should accept and the outputs that it should produce.

Developing an Algorithm

It is essential to device a solution before writing a program code for a given problem. The solution is represented in natural language and is called an algorithm. We can imagine an algorithm like a very well-written recipe fora dish, with clearly defined steps that, if followed, one will end up preparing the dish. We start with a tentative solution plan and keep on refining the algorithm until the algorithm is able to capture all the aspects of the desired solution. For a given problem, more than one algorithm is possible and we have to select the most suitable solution. The algorithm is discussed in section 4.3.

Coding

After finalising the algorithm, we need to convert the algorithm into the format which can be understood by the computer to generate the desired solution. Different high level programming languages can be used for writing a program. It is equally important to record the details of the coding procedures followed and document the solution. This is helpful when revisiting the programs at a later stage. Coding is explained in detail in section 4.8.

Testing and Debugging

The program created should be tested on various parameters. The program should meet the requirements of the user. It must respond within the expected time. It should generate correct output for all possible inputs. In the presence of syntactical errors, no output will be obtained. In case the output generated is incorrect, then the program should be checked for logical errors, if any. Software industry follows standardised testing methods like unit or component testing, integration testing, system testing, and acceptance testing while developing complex applications. This is to ensure that the software meets all the business and technical requirements and works as expected. The errors or defects found in the testing phases are debugged or rectified and the program is again tested. This

Algorithm

In our day-to-day life we perform activities by following certain sequence of steps. Examples of activities include getting ready for school, making breakfast, riding a bicycle, wearing a tie, solving a puzzle and so on. To complete each activity, we follow a sequence of steps. Suppose following are the steps required for an activity ‘riding a bicycle’:

1) remove the bicycle from the stand,
2) sit on the seat of the bicycle,
3) start peddling,
4) use breaks whenever needed and
5) stop on reaching the destination.

Hence, it is clear that we need to follow a sequence of steps to accomplish the task. Such a finite sequence of steps required to get the desired output is called an algorithm. It will lead to the desired result in a finite amount of time, if followed correctly. Algorithm has a definite beginning and a definite end, and consists of a finite number of steps.

Why do we need an Algorithm?

A programmer writes a program to instruct the computer to do certain tasks as desired. The computer then follows the steps written in the program code. Therefore, the programmer first prepares a roadmap of the program to

Characteristics of a good algorithm

- Precision — the steps are precisely stated or defined.
- Uniqueness — results of each step are uniquely defined and only depend on the input and the result of the preceding steps.
- Finiteness — the algorithm always stops after a finite number of steps.
- Input — the algorithm receives some input.
- Output — the algorithm produces some output.

While writing an algorithm, it is required to clearly identify the following:

- The input to be taken from the user
- Processing or computation to be performed to get the desired result
- The output desired by the user

Flowchart

Visual Representation of Algorithms A flowchart is a visual representation of an algorithm. A flowchart is a diagram made up of boxes, diamonds and other shapes, connected by arrows. Each shape represents a step of the solution process and the arrow represents the order or link among the steps.

Table 4.1 Shapes or symbols to draw flow charts

Flowchart symbol	Function	Description
	Start/End	Also called "Terminator" symbol. It indicates where the flow starts and ends.
	Process	Also called "Action Symbol," it represents a process, action, or a single step.
	Decision	A decision or branching point, usually a yes/no or true/ false question is asked, and based on the answer, the path gets split into two branches.
	Input/Output	Also called data symbol, this parallelogram shape is used to input or output data
	Arrow	Connector to show order of flow between shapes.

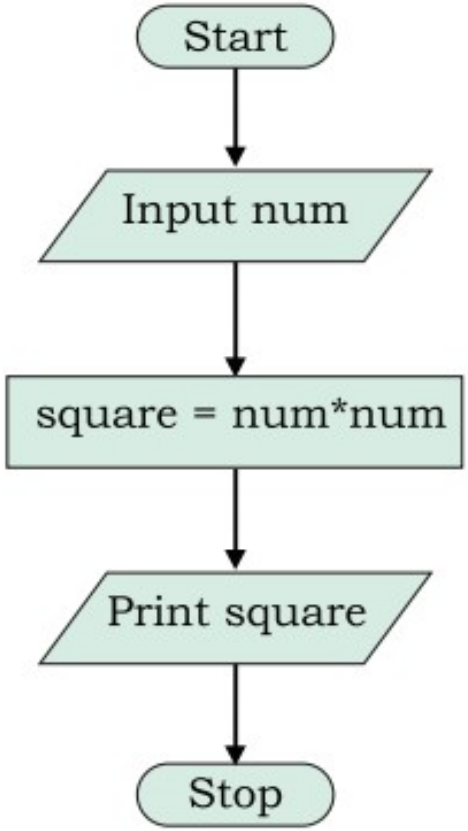
© NCERT not to be republished

Example 4.1: Write an algorithm to find the square of a number.

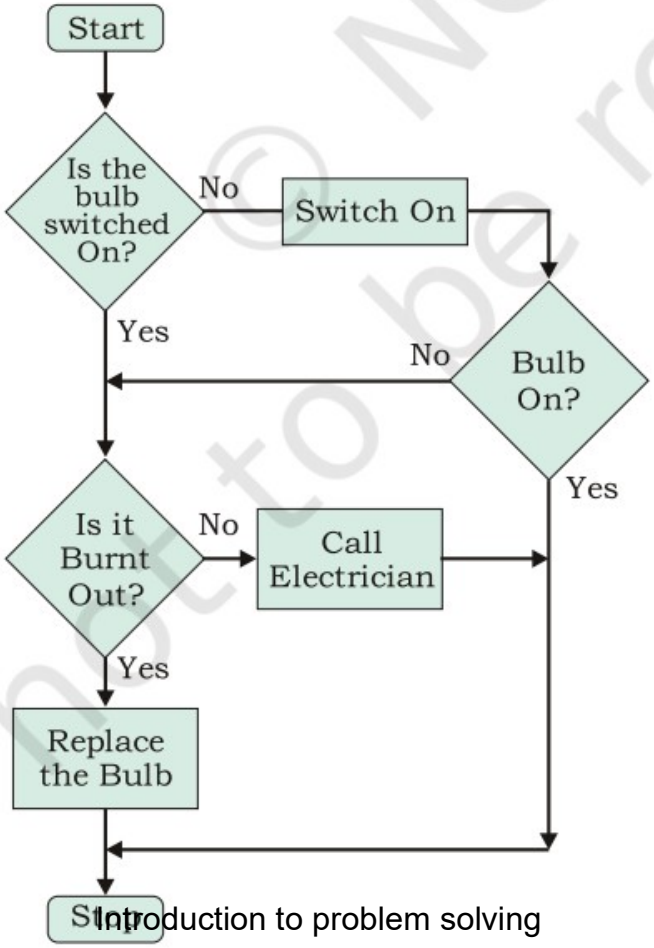
Before developing the algorithm, let us first identify the input, process and output:

- Input: Number whose square is required
- Process: Multiply the number by itself to get its square
- Output: Square of the number
- Algorithm to find square of a number.
- Step 1: Input a number and store it to num
- Step 2: Compute num * num and store it in square
- Step 3: Print square

The algorithm to find square of a number can be represented pictorially using flowchart as shown in Figure 4.2.



Example 4.2: Draw a flowchart to solve the problem of a non-functioning light bulb



Pseudocode

A pseudocode (pronounced Soo-doh-kohd) is another way of representing an algorithm. It is considered as a non-formal language that helps programmers to write algorithm. It is a detailed description of instructions that a computer must follow in a particular order. It is intended for human reading and cannot be executed directly by the computer. No specific standard for writing a pseudocode exists The word “pseudo” means “not real,” so “pseudocode” means “not real code”.

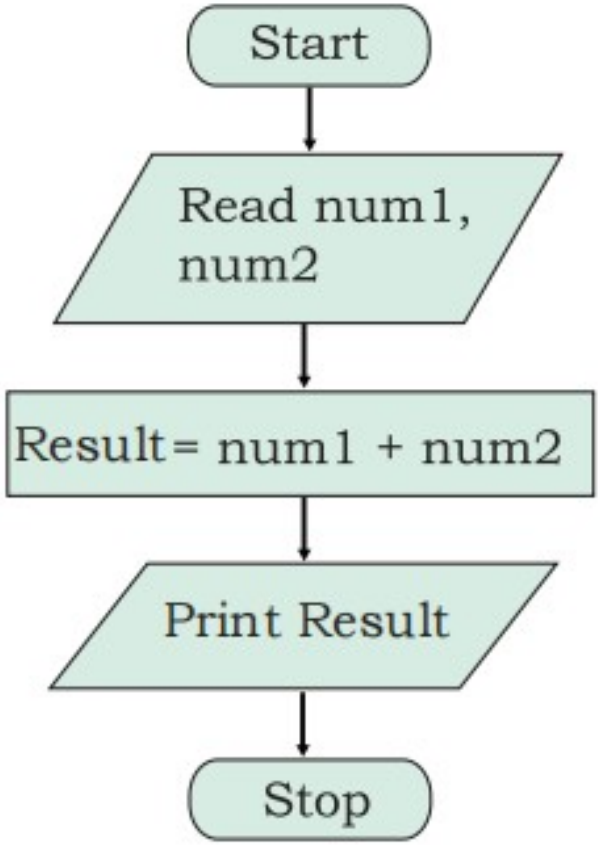
Following are some of the frequently used keywords while writing pseudocode:

- INPUT
- COMPUTE
- PRINT
- INCREMENT
- DECREMENT
- IF/ELSE
- WHILE
- TRUE/FALSE

Example 4.3: Write an algorithm to display the sum of two numbers entered by user, using both pseudocode and flowchart.

- Pseudocode for the sum of two numbers will be:
- input num1
- input num2
- COMPUTE Result = num1 + num2
- PRINT Result

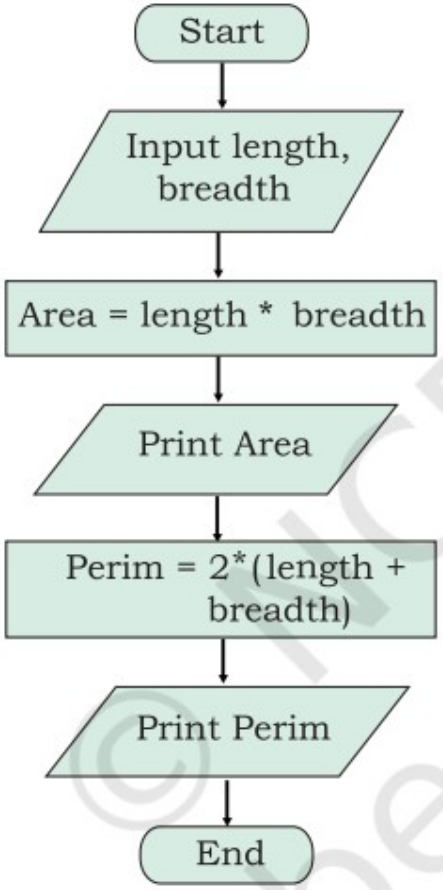
The flowchart for this algorithms is given in Figure 4.4.



Pseudocode for calculating area and perimeter of a rectangle.

```
input length
input breadth
compute Area = length * breadth
print Area
compute Perim = 2 * (length + breadth)
print Perim
```

The flowchart for this algorithm is given in Figure 4.5.



BASIC CONCEPTS OF OBJECTS ORIENTED PROGRAMMING

1. Objects
2. Classes
3. Data abstraction and encapsulation
4. Inheritance
5. Polymorphism
6. Dynamic binding
7. Message passing

OBJECTS

Objects are the basic run-time entities in an object-oriented system. They may represent a person, a place, a bank account, a table of data or any item that the program must handle.

The fundamental idea behind object oriented approach is to combine both data and function into a single unit and these units are called objects. The term objects means a combination of data and program that represent some real word entity. For example: consider an example named Amit; Amit is 25 years old and his salary is 2500.

The Amit may be represented in a computer program as an object. The data part of the object would be (name: Amit, age: 25, salary: 2500)

The program part of the object may be collection of programs (retrive of data, change age, change of salary). In general even any user –defined type-such as employee may be used. In the Amit object the name, age and salary are called attributes of the object.

CLASS

A group of objects that share common properties for data part and some program part are collectively called as class.

In C ++ a class is a new data type that contains member variables and member functions that operate on the variables.

DATA ABSTRACTION

Abstraction refers to the act of representing essential features without including the back ground details or explanations. Classes use the concept of abstraction and are defined as size, width and cost and functions to operate on the attributes.

DATA ENCAPSALATION

The wrapping up of data and function into a single unit (called class) is known as encapsulation. The data is not accessible to the outside world and only those functions which are wrapped in the class can access it. These functions provide the interface between the objects data and the program.

INHERITENCE

Inheritance is the process by which objects of one class acquire the properties of another class. In the concept of inheritance provides the idea of reusablity. This mean that we can add additional features to an existing class with out modifying it. This is possible by desining a new class will have the combined features of both the classes.

POLYMORPHISIM

Polymorphism means the ability to take more than one form. An operation may exhibit different instance. The behaviour depends upon the type of data used in the operation.

A language feature that allows a function or operator to be given more than one definition. The types of the arguments with which the function or operator is called determines which definition will be used.

Overloading may be operator overloading or function overloading.

It is able to express the operation of addition by a single operater say '+'. When this is possible you use the expression x + y to denote the sum of x and y for many different types of x

DYNAMIC BINDING

Binding refers to the linking of a procedure call to the code to the executed in response to the call. Dynamic binding means the code associated with a given procedure call is not known untill the time of the call at run-time. It is associated with a polymorphic reference depends upon the dynamic type of that reference.

MESSAGE PASSING

An object oriented program consists of a set of objects that communicate with each other.

A message for an object is a request for execution of a procedure and therefore will invoke a function (procedure) in the receiving object that generates the desired result. Message passing involves specifying the name of the object, the name of the function (message) and information to be sent.

BENEFITS OF OOP

Oop offers several benefits to both the program designer and the user. Object-oriented contributes to the solution of many problems associated with the development and quality of software products.

The principal advantages are :

1.Through inheritance we can eliminate redundant code and extend the use of existing classes.

2. We can build programs from the standard working modules that communicate with one another, rather than having to start writing the code from scratch. This leads to saving of

development time and higher productivity.

3. This principle of data hiding helps the programmer to build secure programs that can't be invaded by code in other parts of the program.

4. It is possible to have multiple instances of an object to co-exist with out any interference.

5. It is easy to partition the work in a project based on objects.

6. Object-oriented systems can be easily upgraded from small to large systems.

APPLICATION OF OOP

The most popular application of oops up to now, has been in the area of user interface design such as windows. There are hundreds of windowing systems developed using oop techniques.

Real business systems are often much more complex and contain many more objects with complicated attributes and methods. Oop is useful in this type of applications because it can simplify a complex problem.

The promising areas for application of oop includes.

1. Real – Time systems.
2. Simulation and modeling
3. Object oriented databases.
4. Hypertext,hypermedia and expertext.
5. AI and expert systems.
6. Neural networks and parallel programming.
7. Dicision support and office automation systems.
8. CIM / CAM / CAD system.

Algorithms

Algorithm

A set of unambiguous instructions for solving a problem or subproblem in a finite amount of time using a finite amount of *data*

Abstract Step

An algorithmic step containing unspecified details

Concrete Step

An algorithm step in which all details are specified

Basics of C++

C ++ is an object oriented programming language, C ++ was developed by Jarney Stroustrup at AT & T Bell lab, USA in early eighties. C ++ was developed from c and simula 67 language. C ++ was early called ‘C with classes’.

C++ Comments:

C++ introduces a new comment symbol //(double slash). Comments start with a double slash symbol and terminate at the end of line. A comment may start any where in the line and what ever follows till the end of line is ignored. Note that there is no closing symbol.

The double slash comment is basically a single line comment. Multi line comments can be written as follows:

```
// this is an example of
// c++ program
// thank you
```

The c comment symbols /* ….*/ are still valid and more suitable for multi line comments.

```
/* this is an example of c++ program */
```

C++ "Hello, World!" Program

A "Hello, World!" is a simple program that outputs Hello, World! on the screen. Since it's a very simple program, it's often used to introduce a new programming language to a newbie.

Let's see how C++ "Hello, World!" program works.

If you haven't already set up the environment to run C++ on your computer, visit Install C++ on Your Computer.

C++ "Hello World!" Program

```
// Your First C++ Program

#include <iostream>

int main() {
std::cout << "Hello World!";
 return 0;
}
```

Working of C++ "Hello World!" Program

// Your First C++ Program

In C++, any line starting with // is a comment. Comments are intended for the person reading the code to better understand the functionality of the program. It is completely ignored by the C++ compi

#include <iostream>

The #include is a preprocessor directive used to include files in our program. The above code is including the contents of the iostream file.

This allows us to use cout in our program to print output on the screen.

For now, just remember that we need to use #include <iostream> to use cout that allows us to print output on the screen.

int main() {...}

A valid C++ program must have the main() function. The curly braces indicate the start and the end of the function.

The execution of code beings from this function.

std::cout << "Hello World!";

std::cout prints the content inside the quotation marks. It must be followed by << followed by the format string. In our example, "Hello World!" is the format string.

Note: ; is used to indicate the end of a statement.

return 0;

The return 0; statement is the "Exit status" of the program. In simple terms, the program ends with this statement.

Things to take away

We use std:cout in order to print output on the screen.

We must include iostream if we want to use std::cout.

The execution of code begins from the main() function. This function is mandatory. This is a valid C++ program that does nothing.

Watchout for more !!!